13.2 Exemplary AI Failure

If you use a coding wizard such as GitHub Copilot, the AI tool constantly makes code suggestions. If you type `for`, the wizard suggests a loop over a data structure that appears in the previous lines. Entering `//` or `#` gets completed to a full comment by the AI tool, and so on. It's clear that only a fraction of these unsolicited suggestions are useful.

It's up to you to ignore unsuitable suggestions and avoid being distracted if possible. In this section, we don't consider inappropriate suggestions to be errors; we don't want to accuse the AI of not being clairvoyant. (You often get this impression, but it's deceptive.) This section deals with "real errors" in which AI tools provide incorrect or misleading answers despite clear prompts.

13.2.1 Outdated Information

When we were writing this book, questions about the OpenAI API were the most reliable way to go straight to errors. OpenAI has fundamentally changed its API after the cut-off for the training of ChatGPT 4o. This led to the absurd situation that ChatGPT was unable to provide correct information on the use of the parent company's API. (Most other language models were also overwhelmed because it usually takes several months between the start of training for a language model and its publication. By the time you read this book, the situation will almost certainly have improved—unless the API changes again, that is.)

Prompt: I need a Python script to use the OpenAI API as a chat system for programming. The output should be formatted to be readable in a terminal (colors are OK, but no Markdown syntax). In addition, the language model should favor short answers (2 paragraphs max) and code only if possible.

> Certainly! Here's a Python script to use the OpenAI API as a chat system for programming assistance. The output will be formatted for terminal readability, with colors for clarity.
>
> First, ensure you have the `openai` and `termcolor` libraries installed:
>
> ```
> pip install openai termcolor
> ```
>
> Then, you can use the following script:
>
> [code and more details]

We've tried the code, but it doesn't work. The `openai` module doesn't recognize the classes and methods suggested by ChatGPT because they were restructured and renamed after the model was trained. We draw ChatGPT's attention to the error.

Prompt: The code doesn't work, it's not compatible to 1.0 API. Can you help?

> Sure, let's update the script to be compatible with OpenAI's latest API version, which uses the `openai.ChatCompletion.create` endpoint.
>
> [code and more details]

The new code doesn't work any better than the one in the first answer. In a way, you can't blame the AI tool here. The training material was compiled in October 2023. One month later, the OpenAI API was fundamentally changed as part of the 1.0 release. Ultimately, the timing in this example was simply bad luck.

Prompt: What is your training cut-off date?

> My training data includes information up until October 2023. Any events or developments that have occurred after that date aren't within my knowledge base.

Well, we could point ChatGPT to the current documentation:

Prompt: I need a Python script to use the OpenAI API as a chat system . . .

Please note that the API has changed since your training. Up-to-date documentation is available here:

https://platform.openai.com/docs/guides/text-generation

> Sure, here is a Python script that uses the OpenAI API to create a chat system for programming, with a preference for short answers and code only.
>
> [code and more details]

The code remains incorrect even at the third attempt. What is remarkable about the example is the confidence with which ChatGPT introduces false answers (“Certainly,” “Sure,” etc.). At the last prompt, there is no mention in the reply that the link to the current documentation was simply ignored.

13.2.2 Hallucinations

When language models answer questions for which there is no reliable information, they sometimes invent answers. In the context of program development, AI tools suggest the use of modules, libraries, or functions that don’t even exist.

Hallucinations occur because language models use statistical procedures to generate the answer; that is, they continue a text as plausibly as possible once it has been started. It can happen, for example, that a function available in programming language X appears in the answer that actually refers to language Y. Common language models are so “smart” that they adapt the function name to the conventions of the respective programming language (in PowerShell, e.g., to the verb-noun terminology, such as `Get-ClusterNameObject`).

Hallucinations happen most frequently in peripheral IT areas where there is little training material. As long as you use AI tools for coding support with common languages (Python, C, JavaScript) and on common topics (web interfaces, databases, GUI design), it’s extremely unlikely that you’ll encounter hallucinations.

You can find some tips on the internet on how to recognize hallucinations. In our opinion, however, these recommendations aren't very effective. Test the code! If this doesn't work due to nonexistent functions, modules, and so on, search the web for the function names in the traditional way. This usually settles the case. You can ask the AI tool of your choice to make improvements, but if there is insufficient training material, the chances of functioning code aren't particularly high, even on the second or third attempt.

The following example was created using ChatGPT 3.

Prompt: In a Bash script, I use PHP to generate a bcrypted hash code of a password:

```
PW='topSecret'
echo "<?php echo password_hash('$PW',
                                PASSWORD_DEFAULT);" |
php
```

Is there a way to generate the hash code without calling PHP?

> Yes, you can generate the `bcrypt` hash of a password using the `bcrypt` command-line tool, which is available on most Unix-like systems.
>
> The syntax to generate a `bcrypt` hash code of a password using the `bcrypt` tool is:
>
> ```
> bcrypt <password>
> ```
>
> [more details]

The problem is—there is no `bcrypt` command on Linux. Current ChatGPT versions are smarter and recommend the use of the widely used `htpasswd` command.

> Yes, you can generate `bcrypt` hash codes without calling PHP by using other tools that support `bcrypt` hashing. One such tool is `htpasswd`, which is part

```
sudo apt-get install apache2-utils
htpasswd -nbBC 10 user "$PW" | cut -d ":" -f 2 [more
details]
```

13.2.3 Logic Errors

One of the worst errors we encountered while working on this book was a mathematically incorrect optimization of code. The son of one of the three authors had found a Python script on GitHub that calculates the behavior of a mountain bike shock (*four bar linkage suspension*). This script uses the `numpy` and `scipy` modules to perform various calculations that neither father nor son really understood, but whose graphical visualization looked correct. Without any real understanding of the mathematical principles, it was possible to adapt the script to the specific requirements with the help of AI. So far, so good.

However, the resulting code was quite confusing. The AI tool will probably be able to improve that, right?

Prompt: Is there a way to further optimize the code?

[270 lines of Python code]

> Yes, there are several ways to optimize the code further. We can focus on improving readability, reducing redundancy, and enhancing efficiency. Here's a refactored version of the code with these improvements:
>
> - **Minimize Redundant Calculations:** Avoid recalculating the same values multiple times.
> - **Use Functions for Repeated Code:** Create functions for repeated code blocks.
> - **Vectorize Operations:** Where possible, use vectorized operations with NumPy to improve performance.
> - **Enhance Readability:** Use more descriptive variable names and add comments for clarity. Here's the optimized code:

> [200 lines of Python code]

The AI tool, in this case it was ChatGPT 4o, makes an honest effort to re-structure the code, and it does a lot of things right. In particular, it was possible to swap out a code block that occurs several times to a function. This section of code was then shorter and easier to read. Perfect!

However, ChatGPT didn't stop at the mathematical functions of the program and also tried to simplify their code, for example:

```
# Jacobian of vector loop function, before
optimization
def J_4bar(x, l1, l2, l3, l4, theta1, theta4):
    theta2 = x[0]
    theta3 = x[1]
    J = np.zeros((2, 2))
    J[0, 0] = l1 * np.cos(theta1) - l2 *
np.sin(theta2) +
              l3 * np.cos(theta3) + l4 *
np.cos(theta4)
    J[0, 1] = l1 * np.cos(theta1) + l2 *
np.cos(theta2) -
              l3 * np.sin(theta3) + l4 *
np.cos(theta4)
    J[1, 0] = l1 * np.sin(theta1) + l2 *
np.cos(theta2) +
              l3 * np.sin(theta3) + l4 *
np.sin(theta4)
    J[1, 1] = l1 * np.sin(theta1) + l2 *
np.sin(theta2) +
              l3 * np.cos(theta3) + l4 *
np.sin(theta4)
    return J

# Jacobian of vector loop function, after optimization
def J_4bar(x, l1, l2, l3, l4, theta1, theta4):
    theta2, theta3 = x
    return np.array([
        [-l2 * np.sin(theta2), -l3 * np.sin(theta3)],
        [ l2 * np.cos(theta2),  l3 * np.cos(theta3)]
    ])
```

The new code is undoubtedly much shorter, but some of the original parameters (`l1`, `l4`, `theta1`, and `theta4`) are no longer included in the calculation. Depending on the values of these parameters, the result of the optimized function may be completely wrong. It remains a mystery how ChatGPT came up with this "optimization."

What else went wrong with this example?

- The (mathematical) knowledge was insufficient not only for ChatGPT but also for its users. It's never a good idea to ensure the plausibility of the AI results if your experience or expertise is insufficient.
- The script to be optimized was already quite large at approximately 10 KB, at the limit of what common language models (as of the time of writing) can still process well in their entirety. It would have been safer to optimize parts of the code piece by piece.
- The instruction "optimize this code" was rather unspecific. *What* should be optimized? The performance? The functionality? The structure and readability of the code?

13.2.4 Security Issues

Perhaps the greatest danger comes from code that works but has inherent security issues. Such code isn't noticeable because it successfully passes the usual functional tests.

The underlying problem is that the internet is teeming with code snippets that have little regard for security. Such code flows into the training material of language models in the same way as "good" code in terms of security. On average, AI tools deliver equally good or equally bad code as can be found on the internet or in other training sources.

To be fair, it must be said that the use of AI tools does little to change whether a program is secure or insecure. Even many developers or programmers who are brilliant in their specialist areas lack an eye for safety implications. Ultimately, you need to have your code reviewed by security experts, whether written manually or with the help of AI. You must verify its behavior for incorrect or manipulated inputs using fuzzy tests and other methods.

We also want to illustrate the security issue with an example. As a starting point for a REST API that we wanted to create in Python, we asked ChatGPT to create a minimal Docker setup. We've shortened the explanations in the AI response.

Prompt: I want to have a minimal setup to test a REST API running in a Docker container.

1. Apache or Nginx as proxy
2. Docker container (Python) with hello world API, `GET /` should return `{"hello": "world"}`

As little code as possible. No setup for HTTPS/SSL keys (perhaps later).

> To set up a minimal environment for testing a REST API running in a Docker container, we'll use Python with Flask for the API and Nginx as the proxy.
>
> **Step 1:** Create the Flask API. Create a directory for your project, e.g., `hello-world-api`. Inside this directory, create a file named `app.py` with the following content:
>
> ```
> from flask import Flask, jsonify
>
> app = Flask(__name__)
> @app.route('/')
> def hello_world():
>     return jsonify({"hello": "world"})
>
> if __name__ == '__main__':
>     app.run(host='0.0.0.0', port=5000)
> ```
>
> **Step 2:** In the same directory, create a `Dockerfile`:
>
> ```
> # Use the official Python image from the Docker Hub
> FROM python:3.9-slim
> # Set the working directory
> WORKDIR /app
> # Copy the current directory contents into the
> ```

```
container
# at /app
COPY . /app
# Install Flask
RUN pip install Flask
# Make port 5000 available to the world outside this
container
EXPOSE 5000
# Run app.py when the container launches
CMD ["python", "app.py"]
```

Step 3: Create a `docker-compose.yml` file in the same directory:

```
version: '3'
services:
  web:
    build: .
    ports:
      - "5000:5000"
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - web
```

Step 4: Create a `nginx.conf` file in the same directory with the following content:

```
events { }
http {
  server {
    listen 80;
    location / {
      proxy_pass http://web:5000;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For
        $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
      }
    }
  }
```

Step 5: Navigate to your project directory in the terminal and run the following command to build and start your containers:

```
docker-compose up --build
```

Step 6: Open your browser and navigate to *http://localhost*. You should see the JSON response:

```
{
  "hello": "world"
}
```

This minimal setup ensures that your REST API is running in a Docker container with Nginx acting as a reverse proxy. You can later extend this setup to include HTTPS/SSL if needed.

Basically, the answer is great for *fast prototyping*! Assuming a functioning Docker environment, the Hello World API runs in five minutes and provides a perfect starting point for further experiments. So, what's the problem? Unfortunately, there isn't just one, but rather an entire host of security issues:

- ChatGPT has opted for the totally outdated Python 3.9 (step 2). Version 3.12 was the current version at the time, and the next version 3.13 was about to be completed. For security reasons, you should always use the latest possible software versions.
- Three additional lines in `Dockerfile` are sufficient for the Python program to run with the REST API in the Docker container without root rights (also step 2). The improved `Dockerfile` would look like this (with comments on the changes):

```
FROM python:3.12-slim
# Create custom bentzer and your own group
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
WORKDIR /app
COPY . /app
# Change access rights for app directory
RUN chown -R appuser:appgroup /app
RUN pip install Flask
EXPOSE 5000
# Run Python in the appuser account (instead of
root)
USER appuser
CMD ["python", "app.py"]
```

- Instead of `docker-compose.yml` (step 3), the file name `compose.yaml` is now commonly used. The real problem here, however, is port 5000, which is only used for internal communication between the two services. `ports: 5000:5000` opens the port to the whole world. This setting would already be much safer:

  ```
  # in docker-compose.yaml use port 5000 internally
  only
  ports:
    - "5000"
  ```

  A setup with a separate internal network would be ideal. This would require a few additional lines in `compose.yaml`.
- Port 80 is also opened in `docker-compose.yaml` so that the REST API can be tested. As long as the REST API is only intended for testing purposes (which is clearly the case here), it would be safer to restrict the use of port 80 to `localhost`. This means that the test system can only be used on the local computer, but not from outside via the network.

  ```
  # in docker-compose.yaml allow only local access to
  # port 80
  ports:
    - "localhost:80:80"
  ```

That was a lot of Docker-specific details, which this book isn't about. Rather, the example is intended to make it clear that a perfectly functioning solution provided by AI tools is by no means automatically secure. Not only does this statement apply to Docker setups, but to any type of code or IT instructions that ChatGPT and others spit out. As long as you try out an application locally, security considerations usually only play a subordinate role. However, proper security auditing is essential before productive use! In our opinion, it's even better to pay attention to safety right from the start.

Our Own Fault

We were partly to blame for the fact that the solution provided by ChatGPT was inadequate in terms of security. We've explicitly asked for *as little code as possible*. But we could also have added the sentence: *Please provide a secure setup that can be adapted later for productive usage.* Neither ChatGPT nor other AI chatbots will then provide the perfect setup, but the number of security flaws will drop noticeably.

13.2.5 Less Than Ideal Solutions

There are a number of scientific studies that assess the quality of coding responses using established reference solutions. We particularly liked "Is Stack Overflow Obsolete?" by Samia Kabir et al. (May 2024). It compares ChatGPT's answers to popular coding questions with the top-rated solutions on Stack Overflow. For this purpose, ChatGPT 3.5 was used. The PDF of the paper and the underlying data (including all questions) can be found here:

- **https://dl.acm.org/doi/pdf/10.1145/3613904.3642596**
- **https://github.com/SamiaKabir/ChatGPT-Answers-to-SO-questions/tree/main**

The results of the paper are contradictory:

- **High error rate**
  With more than 500 prompts, ChatGPT's responses contained errors in 52% of cases. This doesn't mean that the answers were completely

wrong, but they weren't completely right either. We've chosen the term "less than ideal" in the title of this section.
As many as 77% of the responses contained redundant or superfluous information.

- **ChatGPT doesn't understand the question**
  The reason for the incorrect answers was often that ChatGPT didn't understand the question correctly.
- **Fewer errors with popular topics**
  Answers are more likely to be correct if established knowledge comes into play. The probability of errors increases with questions about new technologies or versions and with unpopular, rarely asked questions.
- **ChatGPT's answers are more digestible**
  Despite the higher error rate, two thirds of all developers prefer the ChatGPT answers to the solutions on Stack Overflow! This is because the answers are often more coherently argued and formulated in a more polite and positive way.

Needless to say, we want to add our own unscientific stuff to this carefully researched publication:

- **Questionable error rate**
  The error rate seems very high to us. Of course, all language models make mistakes—that's what this chapter is all about. But we haven't experienced that every other answer would be wrong. Perhaps this is because we've been working with the next version of GPT throughout. (By the time you read this book, the generation after the next one may already be here!)
- **Functioning, but not optimal code**
  In our work, the code from ChatGPT often worked, so it wasn't wrong. But it was often possible to improve the code, for example, by using modern/newer functions or by applying more elegant algorithms. The AI code wasn't wrong, but it wasn't always as perfect as good quality code written by human developers. (The benchmark would be code from an experienced developer or a well-trained programmer with several years of training and experience.)
- **Targeted response**
  We see the biggest advantage of ChatGPT or other language models

compared to Stack Overflow in the fact that AI tools respond very specifically to our individual question. In practice, it's relatively rare for a Stack Overflow article to answer your exact question. In fact, a Stack Overflow search often comes up with three or four articles that answer partial aspects of the question. Based on that, we then more or less laboriously put together our own code.

The authors of the study turned the tables for the investigation and incorporated established Stack Overflow questions directly into the prompt. That seems a little unfair to us.

In other words, regardless of all the mistakes the AI tool makes, it still answers exactly the question asked (assuming the wording of the prompt is sufficiently clear). An internet search, whether on Stack Overflow or elsewhere, will find countless more or less relevant information in the wider context of the question.

- **Less would be more**

  We share the criticism of the paper authors that ChatGPT's answers are almost always too lengthy. We don't want to read the solution three times—in the introduction, in the code, and then again in the explanations that follow!

  Admittedly, whether the length of an answer is appropriate or not depends very much on your own level of knowledge. The less prior knowledge, the more helpful the redundancy of the AI text. Personally, we like the AI tool *Claude* from Anthropic better than ChatGPT in this respect: Claude gets to the heart of the matter and is far less communicative.

- **Commercial language models are the benchmark**

  In contrast to the paper authors, we have not only used ChatGPT in this book but also tried other language models (often smaller LLMs executed locally with Ollama). Although we would prefer it if we could write the opposite here: the commercial cloud offerings are more convincing in terms of quality.

Further Reading

There are various sites on the internet that compare the quality and error-proneness of language models during code generation:

- **https://symflower.com/en/company/blog**

- **https://artificialanalysis.ai/leaderboards/models**
- **https://github.com/continuedev/what-llm-to-use**

In our view, the results of the annual Stack Overflow survey (**https://survey.stackoverflow.co/2024/ai#efficacy-and-ethics**) on AI tools are also very interesting. Although three quarters of all developers already use AI tools or want to do so in the near future, the developer community sees two fundamental problems: 66% of survey participants generally have doubts about the quality or accuracy of AI answers, and 63% see the main problem with AI tools as being that they lack an overview of the entire code base (more precisely: *AI tools lack the context of the code base, internal frameworks, and/or company knowledge*).